

PROJECT REVIEW 1 — SLR Parser Generator

Design and initial implementation of an SLR parser generator using LR(0) item sets, closure–GOTO construction, ACTION/GOTO tables, and syntax-directed translation for Three-Address Code (TAC) generation.

Team: 24BCE2760, 24BCE2773, 24BCE2778, 24BCE2819, 24BCE2860

Problem Definition & Scope

Problem Statement

Manual construction of SLR parsing tables is repetitive and error-prone. The project automates LR(0) item set generation, constructs SLR ACTION/GOTO tables via FOLLOW sets, and uses the table to parse input with diagnostics and semantic actions.

Primary Objectives

- Define CFG and language specification formally.
- Generate LR(0) canonical collection via closure and GOTO.
- Construct SLR ACTION and GOTO tables using FOLLOW sets.
- Implement stack-based parser that produces diagnostics for invalid inputs.
- Integrate syntax-directed translation for TAC emission.

Project Scope

SCOPE → Grammar → LR(0) States → SLR Table → Parsing → Semantic Actions → Three-Address Code.

Augmented Grammar & Language Specification

The project targets a small expression language with assignment, binary $+$ and $*$, parentheses, identifiers and numeric literals. We augment the grammar with $S' \rightarrow S$ to produce LR items from an explicit start production.

Productions (numbered)

- 0. $S' \rightarrow S$
- 1. $S \rightarrow \text{id} = E$
- 2. $E \rightarrow E + T$
- 3. $E \rightarrow T$
- 4. $T \rightarrow T * F$
- 5. $T \rightarrow F$
- 6. $F \rightarrow (E)$
- 7. $F \rightarrow \text{id}$
- 8. $F \rightarrow \text{num}$

Symbol Classes

Terminals: id, num, =, +, *, (,), \$

Nonterminals: S, E, T, F

Start symbol: S (augmented S')

LR(0) Items — Closure & GOTO Construction

LR(0) items are productions annotated with a dot marking recognition progress. Our implementation follows the canonical algorithm:

01	02	03
<u>closure(I)</u>	<u>GOTO(I, X)</u>	<u>Canonical Collection</u>
If $A \rightarrow \alpha \cdot B\beta$ is in I, then for every production $B \rightarrow \gamma$ add $B \rightarrow \cdot\gamma$. Repeat until fixed point.	For each item $A \rightarrow \alpha \cdot X\beta$ in I, create item $A \rightarrow \alpha X \cdot \beta$ and compute closure of the result. Each distinct set yields a new state.	Initialize $I_0 = \text{closure}(\{S' \rightarrow \cdot S\})$; repeatedly apply GOTO over all grammar symbols to enumerate states until no new states appear.

Example item: $S \rightarrow \text{id} \cdot = E$ (dot after id) — next expected symbol is '='.

Initial Implementation Progress

Working Component: LR(0) Item Set Generator

Implement Closure and GOTO to construct the canonical LR(0) collection.

Code block showing:

```
struct LR0Item { int prod_id; int dot_pos; }
struct State { set<LR0Item> items; map<Symbol, int> transitions; }

void closure(set<LR0Item>& I) {
    bool changed = true;
    while (changed) {
        changed = false;
        for (auto& item : I) {
            if (item.dot_pos < rhs_len[item.prod_id]) {
                Symbol next = rhs[item.prod_id][item.dot_pos];
                if (is_nonterminal(next)) {
                    for (int prod : productions[next]) {
                        LR0Item new_item = {prod, 0};
                        if (I.find(new_item) == I.end()) {
                            I.insert(new_item);
                            changed = true;
                        }
                    }
                }
            }
        }
    }
}

set<LR0Item> goto_fn(const set<LR0Item>& I, Symbol X) {
    set<LR0Item> J;
    for (auto& item : I) {
        if (item.dot_pos < rhs_len[item.prod_id] &&
            rhs[item.prod_id][item.dot_pos] == X) {
            J.insert({item.prod_id, item.dot_pos + 1});
        }
    }
    closure(J);
    return J;
}

for (int i = 0; i < iterations; i++) { ... }
return I;
```

Sample Run

INPUT: GRAMMAR

$S \rightarrow id = E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id \mid num$

Generating LR(0) item sets...

Productions: 9

[LR(0) States: 16

Closure/GOTO construction: COMPLETE

STATUS: Working component verified

Initial Implementation Progress

Data Structures

Productions stored as structured records (head, rhs list, production id). LR(0) items encoded as (prod-id, dot-position). States represented as sets of items with deterministic transition maps keyed by grammar symbols.

Algorithms Implemented

- `closure()` expands items when a nonterminal follows the dot.
- `goto()` advances the dot for a symbol and computes closure.
- Canonical collection generation loops until no new states are added.
- Pretty-print functions for states and transitions facilitate verification.

❏ Sample run: 9 productions, 16 LR(0) states — closure/GOTO complete (initial verification output).

SLR ACTION / GOTO Table Generation

The SLR table is constructed from the canonical LR(0) states plus FOLLOW sets to place reductions. Rules applied:

Shift

If state I contains $A \rightarrow \alpha \cdot a\beta$ where a is terminal and $\text{GOTO}(I,a)=J$, then $\text{ACTION}[I,a] = \text{shift } J$.

Reduce

If I contains $A \rightarrow \alpha \cdot$ and $A \neq S'$, then for every terminal a in $\text{FOLLOW}(A)$, set $\text{ACTION}[I,a] = \text{reduce by } A \rightarrow \alpha$.

Accept & GOTO

If I contains $S' \rightarrow S \cdot$ then $\text{ACTION}[I,\$] = \text{accept}$. For nonterminals X , if $\text{GOTO}(I,X)=J$ set $\text{GOTO}[I,X]=J$.

ILLUSTRATIVE TABLE FRAGMENT

State	id	+	*	(	\$	E	T	F
0	s1	-	-	s2	-	3	4	5
1	-	-	-	-	r7	-	-	-
3	-	s6	-	-	acc	-	-	-
4	-	r3	s7	-	r3	-	-	-

Reductions are placed using FOLLOW of the production head, which is what distinguishes SLR from plain LR(0) table construction.

Parsing Engine — Conflict Detection & Error Handling

We implemented a conventional shift-reduce parser driver that consults ACTION and GOTO tables and maintains a value stack for semantic attributes.

Parsing Loop

Initialize state stack with [0]. Repeat: read lookahead token t ; $action = ACTION[topState, t]$; execute SHIFT / REDUCE / ACCEPT / ERROR accordingly.

Conflict Detection

On table construction, detect multiple non-equal entries per ACTION cell: SHIFT/REDUCE and REDUCE/REDUCE are recorded and reported. The generator logs conflicts with state id, involved productions, and conflicting terminals.

Error Reporting

When ERROR occurs, parser reports: current state, offending token, expected terminals (derived from non-empty ACTION entries), and a short recovery hint. This diagnostic is designed for undergraduates debugging grammars.

Syntax-Directed Translation & Three-Address Code (TAC)

Semantic actions attach synthesized attributes (place) to nonterminals. During reductions, the generator emits TAC instructions and manages temporary naming and value propagation.

Action Templates

For binary operators we create a new temporary $t = \text{newTemp}()$;
 $\text{emit}(t = \text{left.place op right.place})$; assign resulting place to parent nonterminal.

Representative Semantic Actions

$$E \rightarrow E_1 + T : t = \text{new}(); \text{emit}(t = E_1.\text{place} + T.\text{place}); E.\text{place} = t$$
$$T \rightarrow T_1 * F : t = \text{new}(); \text{emit}(t = T_1.\text{place} * F.\text{place}); T.\text{place} = t$$
$$F \rightarrow \text{id} : F.\text{place} = \text{id.lexeme}$$
$$S \rightarrow \text{id} = E : \text{emit}(\text{id.lexeme} = E.\text{place})$$

Example input: $x = a + b * c \rightarrow \text{TAC: } t_1 = b * c ; t_2 = a + t_1 ; x = t_2$

Verification — Test Plan & Cases

The Review 1 test plan prioritizes coverage of valid, edge, and invalid inputs. Each case specifies input tokenization, expected parser outcome and rationale (precedence, grouping, error type).

#	Category	Input	Expected Result
1	Valid	<code>id = num + id * num</code>	Accepted — operator precedence respected; TAC generated
2	Valid	<code>id = (id + num) * num</code>	Accepted — parentheses override precedence
3	Edge	<code>id = num</code>	Accepted — minimal expression
4	Edge	<code>id = (((num)))</code>	Accepted — nested parentheses
5	Invalid	<code>id = + num</code>	Syntax error at '+' — unexpected token
6	Invalid	<code>id = (num + id</code>	Syntax error — missing ')'
7	Invalid	<code>id = num *</code>	Syntax error — unexpected end after operator